

Teoría de la Computación, 2021-1

Nota de clase 5: Autómatas Finitos con ε -transiciones*

Manuel Soto Romero

28 de abril de 2021
Colegio de Ciencia y Tecnología UACM

5.1. Definición

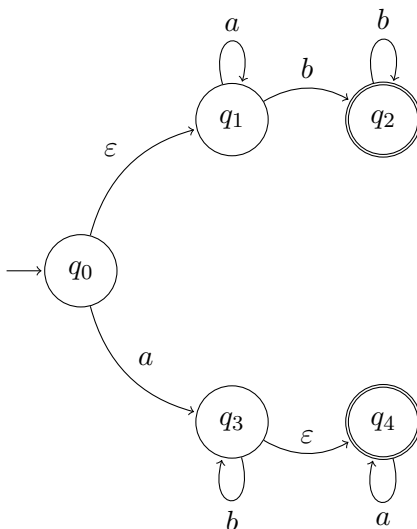
Un *autómata finito con ε -transiciones* (AFN- ε) es un autómata no determinista $M = \langle \Sigma, Q, q_0, F, \delta \rangle$ en el que la función de transición está definida como:

$$\delta : Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow \mathcal{P}(Q)$$

La transición $\delta(q, \varepsilon) = \{p_{i1}, \dots, p_{in}\}$, llamada *transición ε* o *transición espontánea*, tiene el siguiente significado computacional: estando en el estado q , el autómata puede cambiar a uno cualquier de los estados $p_{i1}, \dots, p_{in}$, independientemente del símbolo leído. Dicho de otra manera, las transiciones ε permiten al autómata cambiar internamente de estado sin consumir el símbolo leído.

En el diagrama del autómata, las transiciones ε dan lugar a aristas con etiquetas ε . Una cadena de entrada w es aceptada por un AFN- ε si existe por lo menos una trayectoria, desde el estado q_0 , cuyas etiquetas son exactamente los símbolos de w , intercalados con cero, uno o más ε s.

En los autómatas AFN- ε , al igual que en los AFN, puede haber múltiples caminos para una misma cadena de entrada. Veamos un ejemplo:



*Tomada de [1] con ligeras modificaciones.

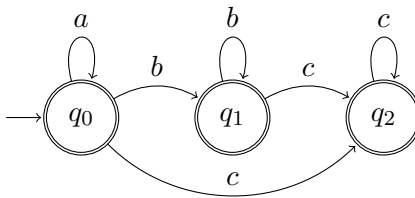
Ejemplos de cadenas aceptadas por este autómata son:

- aab
- $abaa$
- $abbaa$

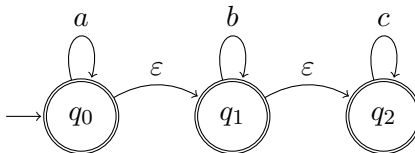
Los AFN- ε nos dan más libertad en el diseño de autómatas, especialmente cuando tenemos varias concatenaciones.

Ejemplo 5.1. $\Sigma = \{a, b, c\}$. $L = a^*b^*c^*$

AFN que acepta a L :



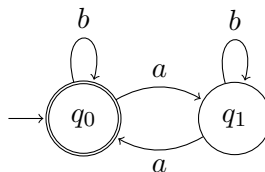
AFN- ε que acepta a L :



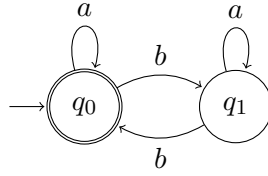
Este autómata es muy similar a la expresión regular $a^*b^*c^*$: las concatenaciones son reemplazadas por transiciones ε .

Ejemplo 5.2. $\Sigma = \{a, b\}$. L = lenguaje de todas las cadenas sobre Σ que tienen un número par de as o un número par de bs.

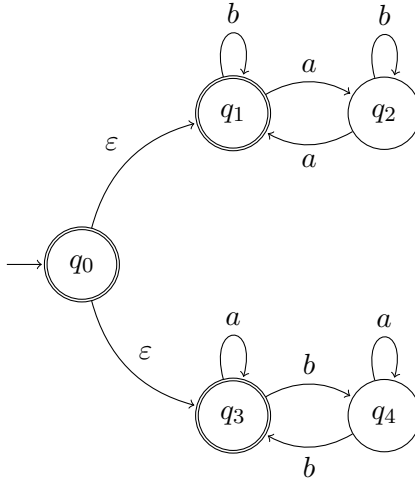
AFD que acepta el lenguaje de las cadenas con un número par de as:



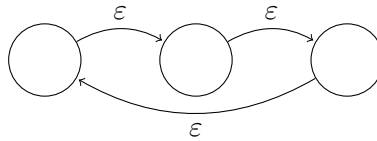
AFD que acepta el lenguaje de las cadenas con un número par de bs:



AFN- ε que acepta el lenguaje de las cadenas con un número par de as o número par de bs :



A diferencia de los AFD y los AFN, en los AFN- ε pueden existir cálculos infinitos, es decir, cálculos que nunca terminan. Esto puede suceder si el autómata ingresa a un estado desde el cual haya varias transiciones ε encadenadas que regresen al mismo estado. Por ejemplo, el siguiente autómata:



Ejercicio 5.1. Diseñar un AFN- ε que acepte el lenguaje $(ab + b)^* ab^*$, sobre $\Sigma = \{a, b\}$.

5.2. Equivalencia entre AFN- ε y AFN

En esta sección mostraremos que el modelo AFN- ε es computacionalmente equivalente al modelo AFN. Es decir, se pueden quitar las transiciones ε y añadir nuevas que las simulen sin alterar el lenguaje aceptado. Comencemos con la siguiente observación.

Observación 5.1. Un AFN puede ser visto como un AFN- ε en el que, simplemente hay *cero* transiciones ε .

Con esta observación, tenemos el siguiente teorema:

Teorema 5.1. Dado un AFN- ε M , se puede construir un AFN M' equivalente a M tal que $L(M) = L(M')$.

La demostración del teorema queda fuera de los alcances de este curso, por lo que en su lugar, presentaremos un método de transformación. Necesitamos primero que nada definir el concepto de ε -cerradura.

Para un estado q , la ε , denotada $ECLOSURE(q)$ es el conjunto de estados de un AFN- ε a los que se puede llegar desde q por 0, 1 o más transiciones ε . Nótar además que $ECLOSURE(q) \neq \delta(q, \varepsilon)$. Por definición $q \in ECLOSURE(q)$. De esta forma, la ε -cerradura de un conjunto de estados $\{q_1, \dots, q_k\}$ se define por:

$$ECLOSURE(\{q_1, \dots, q_k\}) = ECLOSURE(q_1) \cup \dots \cup ECLOSURE(q_k)$$

Además, $ECLOSURE(\emptyset) = \emptyset$.

De esta manera, podemos construir un AFN $M' = \langle \Sigma, Q, q_0, F', \delta' \rangle$ a partir del AFN- ε $M = \langle \Sigma, Q, q_0, F, \delta \rangle$, mediante la siguiente función de transición:

$$\begin{aligned} \delta' : Q \times \Sigma &\rightarrow \mathcal{P}(Q) \\ (q, a) &\rightarrow \delta'(q, a) = ECLOSURE(\delta(ECLOSURE(q), a)) \end{aligned}$$

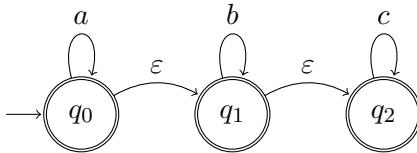
M' simula así las transiciones ε de M teniendo en cuenta todas las posibles trayectorias. F' se define como:

$$F' = \{q \in Q : ECLOSURE(q) \text{ contiene al menos un estado de aceptación}\}$$

Es decir, los estados de aceptación de M' incluyen los estados de aceptación de M y aquellos estados desde los cuales se puede llegar a un estado de aceptación por medio de una o más transiciones ε .

Veamos un ejemplo.

Ejemplo 5.3. Vamos a ilustrar el método anterior con el autómata del Ejemplo 5.1 al que llamaremos M :



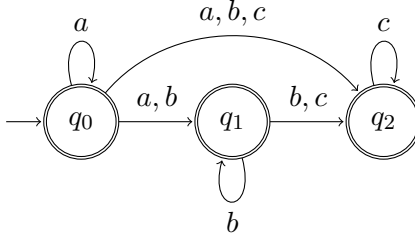
Recordemos que $L(M) = a^*b^*c^*$. Comencemos entonces calculando la ε cerradura de cada estado de nuestro autómata.

$$\begin{aligned} ECLOSURE(q_0) &= \{q_0, q_1, q_2\} \\ ECLOSURE(q_1) &= \{q_1, q_2\} \\ ECLOSURE(q_2) &= \{q_2\} \end{aligned}$$

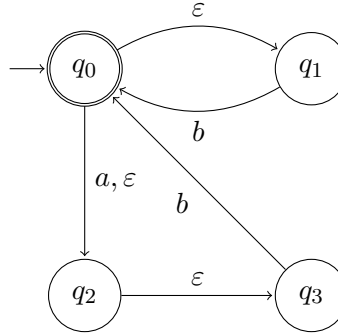
A partir de aquí construimos las transiciones correspondientes, abreviamos *ECLOSURE* como *EC*:

$$\begin{aligned}
\delta'(q_0, a) &= EC(\delta(EC(q_0), a)) = EC(\delta(\{q_0, q_1, q_2\}, a)) = EC(\{q_0\}) = \{q_0, q_1, q_2\} \\
\delta'(q_0, b) &= EC(\delta(EC(q_0), b)) = EC(\delta(\{q_0, q_1, q_2\}, b)) = EC(\{q_1\}) = \{q_1, q_2\} \\
\delta'(q_0, c) &= EC(\delta(EC(q_0), c)) = EC(\delta(\{q_0, q_1, q_2\}, c)) = EC(\{q_2\}) = \{q_2\} \\
\delta'(q_1, a) &= EC(\delta(EC(q_1), a)) = EC(\delta(\{q_1, q_2\}, a)) = EC(\emptyset) = \emptyset \\
\delta'(q_1, b) &= EC(\delta(EC(q_1), b)) = EC(\delta(\{q_1, q_2\}, b)) = EC(\{q_1\}) = \{q_1, q_2\} \\
\delta'(q_1, c) &= EC(\delta(EC(q_1), c)) = EC(\delta(\{q_1, q_2\}, c)) = EC(\{q_2\}) = \{q_2\} \\
\delta(q_2, a) &= EC(\delta(EC(q_2), a)) = EC(\delta(\{q_2\}, a)) = EC(\emptyset) = \emptyset \\
\delta(q_2, b) &= EC(\delta(EC(q_2), b)) = EC(\delta(\{q_2\}, b)) = EC(\emptyset) = \emptyset \\
\delta(q_2, c) &= EC(\delta(EC(q_2), c)) = EC(\delta(\{q_2\}, c)) = EC(\{q_2\}) = \{q_2\}
\end{aligned}$$

El conjunto de estados finales es $\{q_0, q_1, q_2\}$ dado que las cerraduras de todos los estados contienen estados finales. De esta forma, el autómata AFN queda como:



Ejercicio 5.2. Construye un AFN equivalente al siguiente AFN- ε :



5.3. Teorema de Kleene

En las notas y secciones anteriores, se ha mostrado la equivalencia computacional entre los AFD, AFN y AFN- ε , lo cual puede ser descrito de la siguiente forma:

$$AFD \equiv AFN \equiv AFN-\varepsilon$$

Esto quiere decir que para cada uno de estos tipos de autómatas se pueden construir otro tipo de autómatas equivalentes. Por lo tanto, estos tres modelos aceptan exactamente los mismos modelos. El Teorema de Kleene establece que tales lenguajes son precisamente los lenguajes regulares.

Teorema 5.2. *Un lenguaje es regular si y sólo si es aceptado por un autómata finito (AFD o AFN o AFN- ε).*

La demostración de este teorema queda fuera de los alcances del curso. Sin embargo, la misma demostración nos permite analizar la equivalencia que hay entre las expresiones regulares y los autómatas finitos (en ambos sentidos) por medio de dos partes: La Parte I demuestra que dada una expresión regular R sobre un alfabeto Σ , se puede construir un AFN- ε M tal que el lenguaje aceptado por M sea exactamente el lenguaje representado por R . Por otro lado, la Parte II demuestra que para todo AFN M existe una expresión regular R tal que $L(M) = R$. Para los fines de este curso nos enfocaremos en la Parte I de la demostración desde un enfoque algorítmico.

Recordemos que una expresión regular se define recursivamente como:

Sea Σ un alfabeto. La *expresión regular* sobre Σ y los conjuntos que ellas denotan son definidos recursivamente como sigue:

- $\emptyset$ es una expresión regular y denota el conjunto vacío.
- ε es una expresión regular y denota el conjunto $\{\varepsilon\}$.
- Para cada a en Σ , a es una expresión regular y denota el conjunto $\{a\}$.
- Si r y s son expresiones regulares denotando el lenguaje R y S respectivamente entonces $(r + s)$, rs , r^* son expresiones regulares que denotan los conjuntos $R \cup S$, RS y R^* , respectivamente.

De esta forma, a partir de cada una de las construcciones anteriores, podemos construir también autómatas que acepten los lenguajes que representan cada una de éstas. Veamos cada uno de estos casos. Es importante notar que el autómata obtenido será un AFN- ε .

Caso 1 ($\emptyset$)

Autómata

El siguiente autómata acepta el lenguaje $\emptyset$, es decir el lenguaje representado por la expresión regular $R = \emptyset$.



Caso 2 (ε)

Autómata

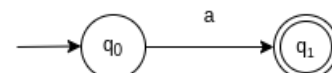
El siguiente autómata acepta el lenguaje $\{\varepsilon\}$, es decir, el lenguaje representado por la expresión regular $R = \varepsilon$.



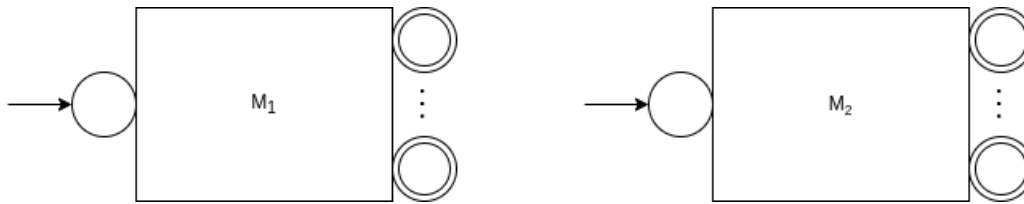
Caso 3 (a)

Autómata

El siguiente autómata acepta el lenguaje $\{a\}$, $a \in \Sigma$, es decir, el lenguaje representado por la expresión regular $R = a$.



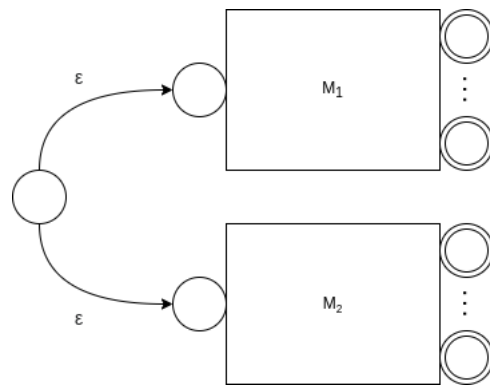
Para los siguientes casos, supóngase que para las expresiones regulares R y S existen AFN- ε M_1 y M_2 tales que $L(M_1) = L(R)$ y $L(M_2) = S$. Esquemáticamente vamos a presentar los autómatas M_1 y M_2 de la siguiente forma:



Caso 4 ($R + S$)

Autómata

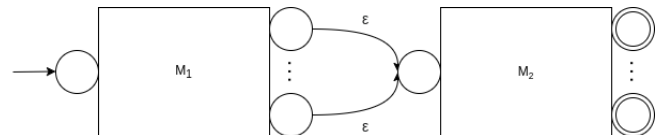
Los autómatas M_1 y M_2 se conectan en paralelo y los estados finales del nuevo autómata son los estados de M_1 junto con los de M_2 .



Caso 5 (RS)

Autómata

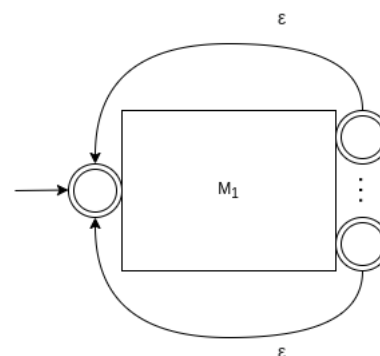
Los autómatas M_1 y M_2 se conectan en serie y los estados finales del nuevo autómata son únicamente los estados finales de M_2 .



Caso 6 (R^*)

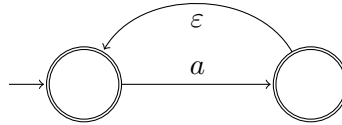
Autómata

Los estados finales del nuevo autómata son los estados finales de M_1 junto con el estado inicial.

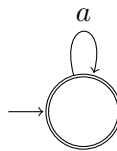


5.3.1. Ejemplo

De acuerdo con las construcciones presentadas, a continuación veremos algunos ejemplos, sin embargo antes de empezar, analicemos el siguiente autómata que acepta el lenguaje a^* :

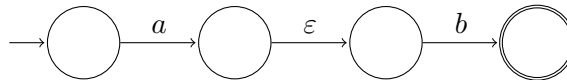


Para simplificar las próximas construcciones, usaremos en su lugar el ciclo de un estado:

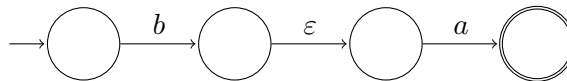


Ejemplo 5.4. Vamos a utilizar el procedimiento anterior para construir un AFN- ϵ que acepte el lenguaje $a^* (ab + ba)^* + a (b + a^*)$ sobre el alfabeto $\{a, b\}$.

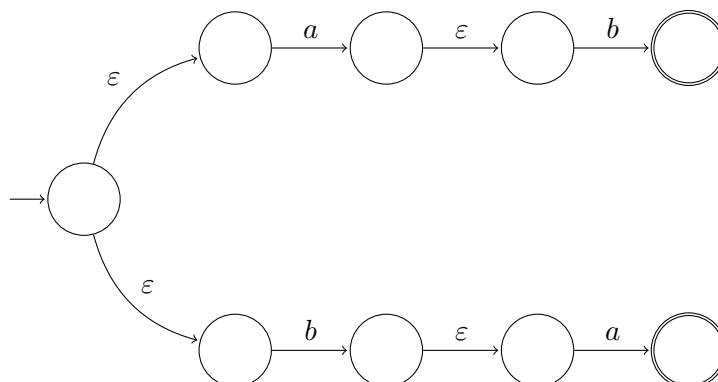
- Autómata que acepta ab :



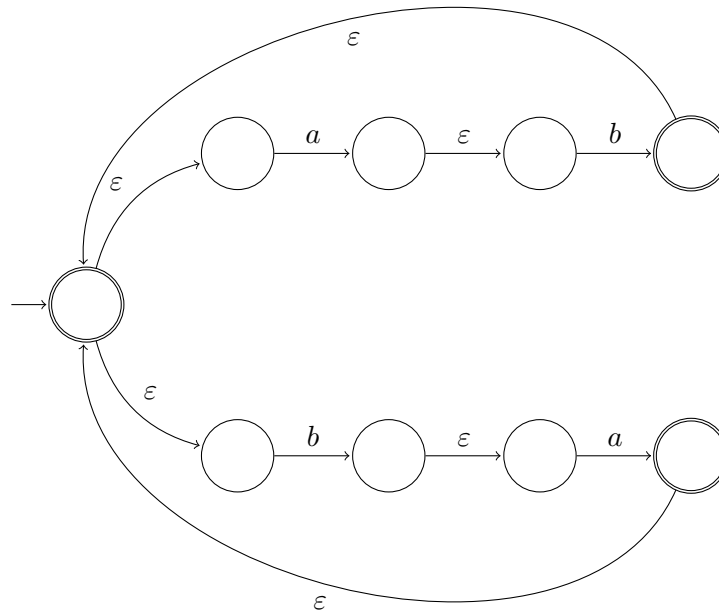
- Autómata que acepta ba :



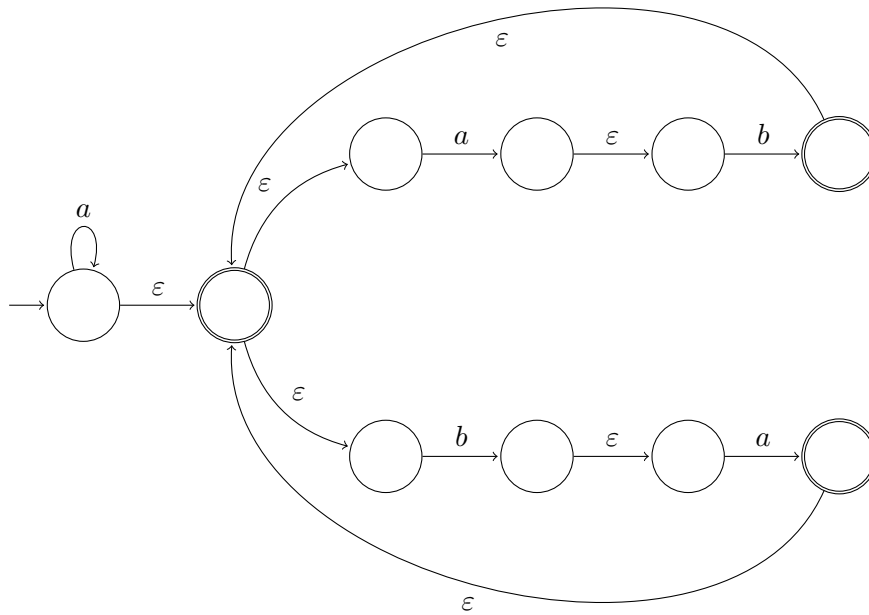
- Autómata que acepta $ab + ba$:



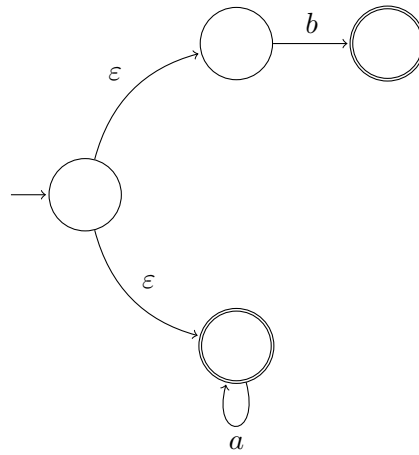
- Autómata que acepta $(ab + ba)^*$:



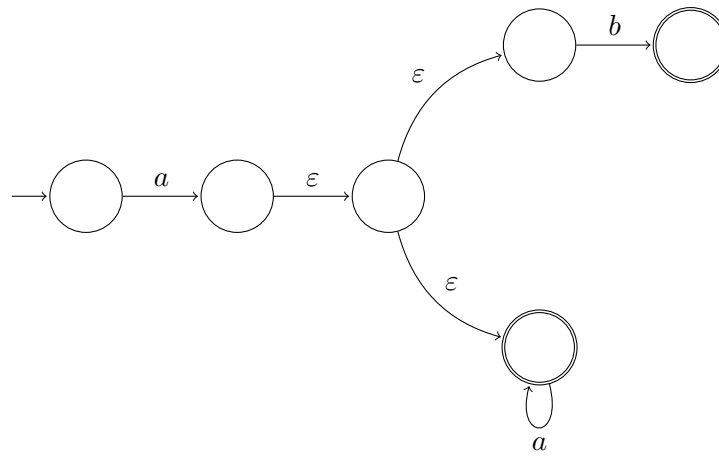
- Autómata que acepta $a^*(ab + ba)^*$:



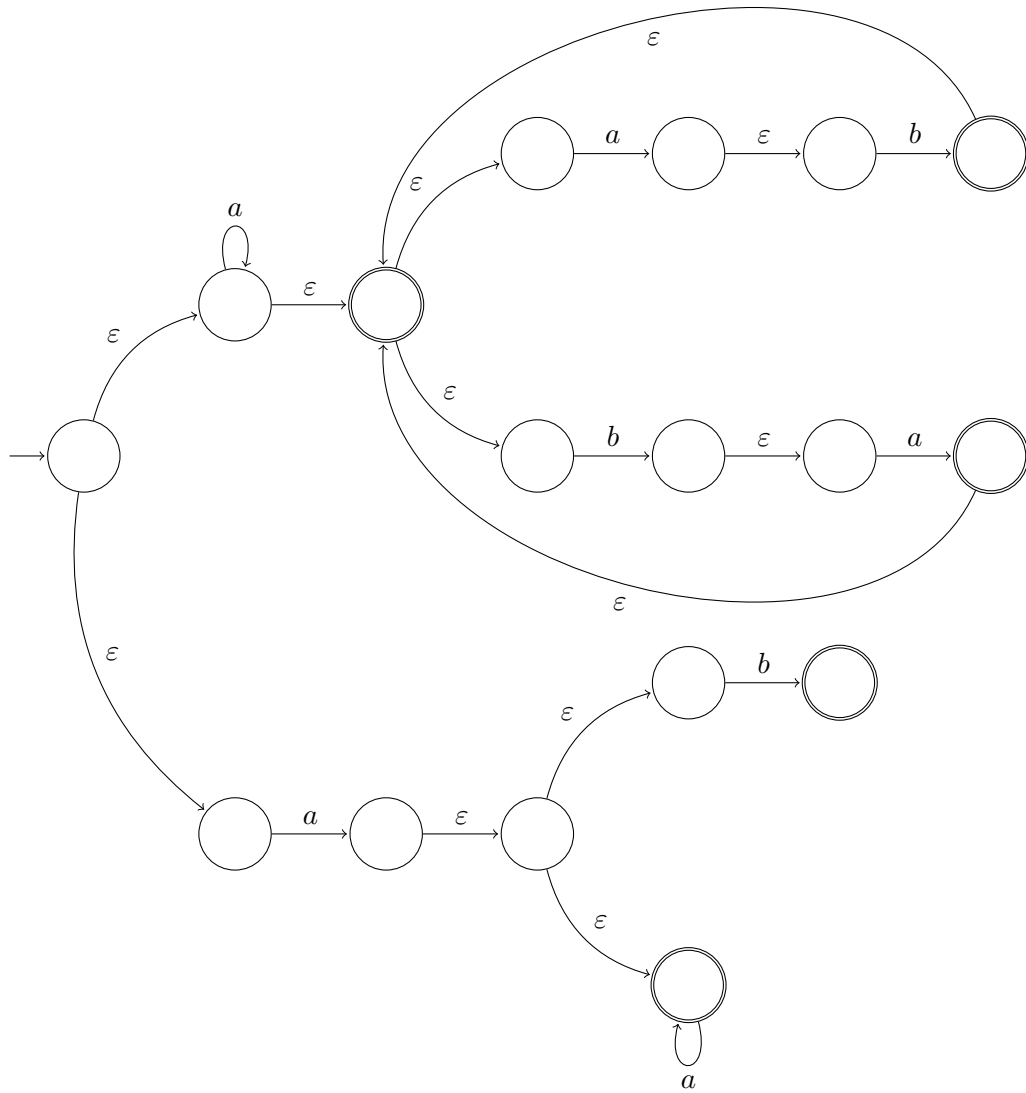
- Autómata que acepta $b + a^*$:



- Autómata que acepta $a(b + a^*)$:



- Autómata que acepta $a^*(ab + ba)^* + a(b + a^*)$:



Ejercicio 5.3. Diseñar autómatas AFN- ϵ que acepten los siguientes lenguajes sobre el alfabeto $\Sigma = \{a, b, c\}$:

- $a^* (b + ab^* + ab^*a) c^* + (a + b) (a + ac)^*$
- $c^* a (a + ba)^* (abc)^* + c^* (a + cb^*c)$

Referencias

- [1] Rodrigo De Castro, *Notas de Clase de Teoría de la Computación: Lenguajes, autómatas, gramáticas*, Universidad Nacional de Colombia, Facultad de Ciencias, 2004.